

Local Volatility, Forward

A lecture on calibrating a triangulated Dupire surface from the parameters up
Vol-Fitter Technical Notes, No. 04

Vol-Fitter

July 27, 2026

Abstract

There are two ways to get a local-volatility surface out of an option market, and they differ by the direction in which the Dupire equation is read. The classical way reads it as a formula: differentiate the implied surface twice in strike and once in maturity, and divide. Differentiation of noisy data is an ill-posed operation, and the denominator is a density — so on the sparse, noisy chains one actually observes, the quotient manufactures spikes and negative variances precisely where the surface is needed most. This lecture develops the alternative in full: treat the equation as a *pricing map* and the surface as the unknown of an inverse problem. Three pieces of mathematics carry everything. A finite-element parametrization — a continuous piecewise-affine sheet on a triangulated strike-maturity grid — reduces positivity of a surface to box bounds on finitely many nodal values, because affine interpolation is a convex combination. A fully implicit discretization of the forward Dupire equation has an M -matrix step, hence a discrete maximum principle with no step-size restriction — and we show, constructively, how Crank–Nicolson loses exactly this property. And because the parametrization enters the PDE linearly, every sensitivity the optimizer needs marches behind the same tridiagonal factorization as the prices, with the true adjoint derived alongside. The inverse problem itself is treated honestly: sparse vanillas do not identify the surface — we exhibit two surfaces 22 vol points apart that reprice the same quotes to 1.6 vol bp RMS — so calibration returns one Tikhonov-regularized representative of an equivalence class, and the residual arbitrage of the discretization is measured, never assumed away. All claims are exercised by a production-grade implementation: synthetic quotes reprice to 0.6 vol bp with the latent surface recovered to 0.25 vol points on the quote-covered region, and a real SPY/NVDA chain fits to 2.4/11.9 vol bp — 15.9/56.3 on a refined operator, the honest metric.

Contents

1	The wrong direction, and the right one	3
2	The equation the surface must obey	5
3	A surface you can hold	6
3.1	Finitely many positive numbers	6
3.2	Beyond the hull: the wing contract	9
4	What the grids must resolve	9
5	Pricing on the lattice	10
5.1	The discrete march	10
5.2	Why not Crank–Nicolson	12
5.3	Reading out, and measuring what leaks	13
6	Variance swaps	14
7	What the data can see, and the objective	14
7.1	Two surfaces, one set of quotes	14
7.2	The objective	15
8	Every derivative from the same factorization	16
8.1	The adjoint, derived and not yet needed	17
9	The cost of a fit	18
10	Case files: two lessons in numerical pathology	19
11	Worked examples	21
11.1	The round trip, and its two separate numbers	21
11.2	The real-chain benchmark	22
12	What is genuinely original here	23
13	Limitations	24
A	Hyperparameter atlas	24
B	Performance notes	25
C	Traceability	26
D	Reference implementation: Dupire extraction	28

1 The wrong direction, and the right one

Everything in this note descends from one formula and one decision about which way to read it. Dupire’s observation [1] is that a full continuum of arbitrage-free call prices determines a unique diffusion consistent with them, with local variance

$$\nu(\tau, x) = \frac{\partial_\tau c}{\frac{1}{2} x^2 \partial_{xx} c} . \quad (1)$$

Read left to right, this is an *extraction*: differentiate the market, then divide. On paper it is exact. The trouble is that no one has a continuum of arbitrage-free prices; a real chain is a few dozen noisy quotes per expiry, interpolated somehow, and (1) divides one second derivative of that interpolant by another. It is worth quantifying why this must fail, because the failure is structural, not bad luck. Numerical differentiation is the canonical ill-posed operation: a quote error of size ε at strike spacing Δk enters a second-difference estimate of $\partial_{kk} w$ at size $\sim 4\varepsilon/\Delta k^2$ — the *finer* your strike grid, the *worse* the noise amplification, an inverse crime with no stable limit. And the denominator it lands in is not just small: $\partial_{xx} c$ is the risk-neutral density, so the first time the amplified noise pushes it through zero, the extraction returns a *negative variance* — an object no PDE, no Monte Carlo, and no risk system downstream can use. Figure 1 performs the experiment.

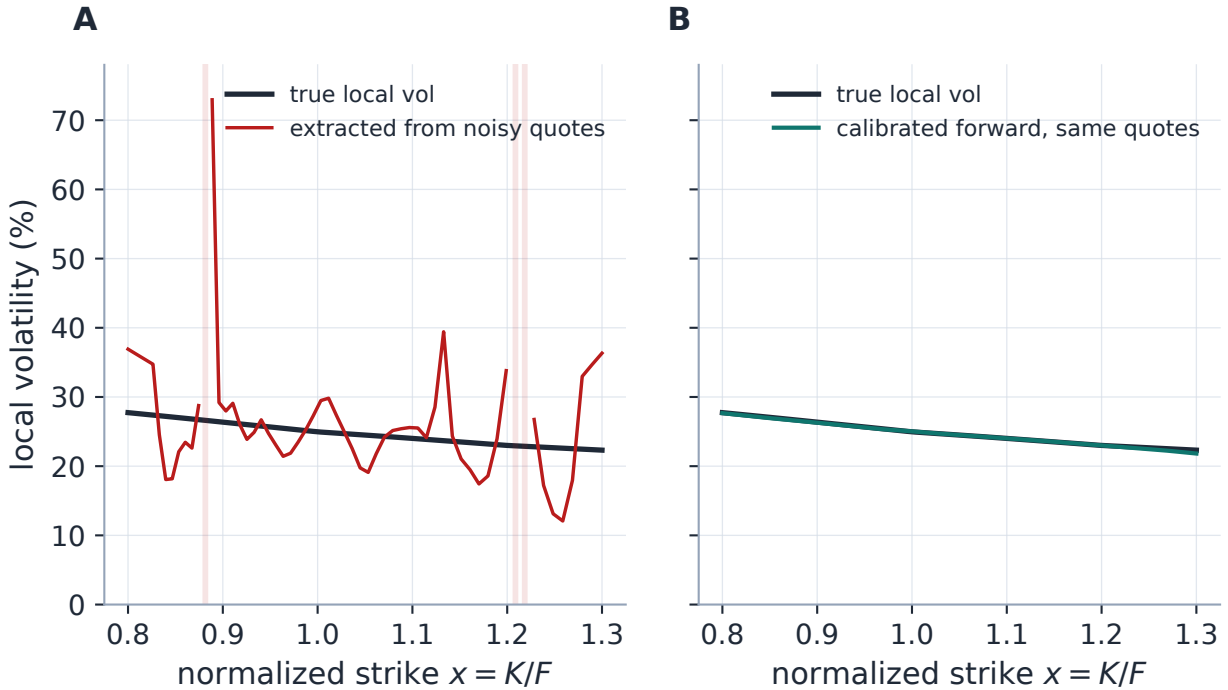


Figure 1: The same market data, both directions. A: local volatility extracted through (1) from 13 quotes per expiry carrying a deterministic ± 50 bp alternating ripple (a bid–ask bounce), linearly interpolated and finite-differenced — the naive pipeline. The extraction swings between 12% and 73% around a true surface near 25%, and at 3 strikes (shaded bands) it fails outright: negative variance, no admissible diffusion. B: the *same* noisy quotes handed to the forward calibration of this note — the fitted surface stays within 0.5 vol points of the truth, because the regularized forward fit averages the noise that the derivative quotient amplifies.

The right direction is to read (1) *backward*, as Andreassen and Høuge taught [2]: decide that the unknown is the surface ν itself, *parametrize* it, price it forward through the Dupire PDE — which only ever differentiates the smooth numerical solution, never the data — and ask the optimizer for the positive surface whose prices meet the quotes. Positivity is then imposed on the parameters, so every iterate of the optimizer already is an admissible diffusion. Readers of Note 01 will recognize the design: it is the LQD philosophy, one dimension up — pick the parametrization in which the constraint set is trivial, and let the pricing map do the hard work.

This lecture builds that machine from first principles: the equation and what it propagates (section 2), the triangulated sheet and why box bounds suffice (section 3), the vertex grid (section 4), the discrete march and its maximum principle (section 5), variance swaps (section 6), what the data identifies and the objective (section 7), the derivatives (section 8), the speed story (section 9), and two case files from production history (section 10). Production protects it with the following invariants.

Invariant

1. Every optimizer iterate is a genuine, strictly positive local-variance surface *on the grid's hull* — positivity is carried by the parameters, never policed by a penalty. (The controlled linear put extrapolation beyond the hull is outside this guarantee; section 3.)
2. The pricing map never differentiates market data: quotes enter only as targets of a forward PDE solve.
3. The discrete prices obey a maximum principle — the scheme is monotone and unconditionally ℓ^∞ -stable. That is stability, not an arbitrage proof: butterfly and calendar cleanliness of the output are *measured* by explicit grid diagnostics, never assumed (section 5.3).
4. What calibration returns is one *regularized representative* of an equivalence class of surfaces the data cannot distinguish — and the note says which regularizer chooses which direction of that representative (section 7.1).

Where this object lives. The slice models of Notes 01–03 fit one expiry at a time and stitch expiries with a calendar constraint. The surface model is the opposite object: *one* diffusion pricing every strike and expiry of a name simultaneously — the natural carrier for scenario transport (Note 12), variance swaps (Note 08), and surface-level extrapolation (Note 14). As a computation it is a loop with four mathematical stages: parametrize the surface, march the PDE forward for prices and sensitivities, stack the residuals of a regularized least-squares functional, and repeat under box bounds until converged — then *re-price on a refined operator*, because a fit can hide discretization error inside its own optimum (section 5.3).

Running characters. Two recur. A *synthetic skewed truth* — a known positive surface, priced through the same PDE and quoted at four expiries — against which the method can be tested with the answer known (fig. 1 already used it). And a *real equity chain* (SPY, with NVDA as the hard case), fitted end to end; the triangulated sheet of fig. 2 is its surface, and its numbers close the lecture (section 11).

The notation ledger. As in Note 01, few symbols, each defined once, and one differentiation convention: a prime is the derivative of a one-variable function, a subscripted ∂ a partial derivative.

$x = K/F$, $c(\tau, x)$	normalized strike; normalized call	τ	variance time (the clock)
$\nu(\tau, x)$	local variance σ_{loc}^2	(τ_i, x_j) , θ_{ij}	grid vertices; nodal variances
ϕ_ℓ , m	hat basis; number of vertices	$[\nu_{\text{lo}}, \nu_{\text{hi}}]$, a	variance box; left-wing slope
Δx , $\Delta \tau$, U^n	lattice steps; marched call values	A , $B = I - \Delta \tau A$	space operator; step matrix
$s_\ell = \partial U / \partial \theta_\ell$	tangent sensitivities	μ^n	adjoint states (theory)
$r_\bullet$, $w_\bullet$, η_q	residual blocks, weights, vega scale	L , λ , ρ	roughness operator, weight, balance
$I(\tau)$	expected integrated variance	$w(k, \tau)$	implied total variance (appendix)

Table 1: Every symbol in the note. The stencil weights $a_i^\pm$ of section 5 and the SSVI-free synthetic truth are local to their sections.

2 The equation the surface must obey

Assumption 1 (Units, and the one clock). Work in forward-normalized units: $x = K/F$ is the normalized strike and $c(\tau, x)$ the normalized call, so deterministic rates, borrow and dividends are absorbed into the forward (Note 06), and the quotes are de-Americanized mids (Note 05). Time is measured throughout on the *variance clock* τ : the monotone event-weighted time of Note 11, equal to calendar time whenever the event clock is off. The nodal parameters are local variances per unit variance time, the PDE marches in τ , and every maturity-like quantity in this note (steps, vertices, the var-swap integral) lives on the τ axis; in calendar time T the same dynamics simply pick up the clock speed, $\partial_T c = \frac{1}{2} \tau'(T) \nu x^2 \partial_{xx} c$. The model is a pure diffusion: no jumps, so a genuine short-dated event spike is only approximated (the clock absorbs *scheduled* events instead).

With the units fixed, the forward Dupire equation is one line:

$$\partial_\tau c(\tau, x) = \frac{1}{2} \nu(\tau, x) x^2 \partial_{xx} c(\tau, x), \quad c(0, x) = (1 - x)^+. \quad (2)$$

A forward *parabolic* equation in the strike variable, with the payoff as initial data: heat flow, with $\frac{1}{2} \nu x^2$ as a spatially varying conductivity. That analogy is not decoration — it is the source of every structural property we will use. Heat flow spreads bumps and never sharpens them; translated back into option language, spreading is exactly the no-arbitrage direction:

Proposition 1 (Positive local variance propagates no-arbitrage). *Let c solve (2) with bounded $\nu \geq 0$. Then for all $\tau > 0$: (i) $\partial_{xx} c(\tau, \cdot) \geq 0$ — no butterfly arbitrage; (ii) $\partial_\tau c(\tau, x) \geq 0$ — the normalized call is non-decreasing in maturity at fixed x . Under deterministic carry, (ii) is equivalent to the familiar fixed-dollar-strike calendar condition: the fixed- K spread maps to the fixed- x one through $K \mapsto x = K/F$ and the discounting absorbed by the normalization (Note 06) — related by that transformation, not identical coordinates.*

Proof sketch. Let $p = \partial_{xx} c$. Differentiating (2) twice in x gives the forward Fokker–Planck-type equation $\partial_\tau p = \frac{1}{2} \partial_{xx}(\nu x^2 p)$, whose initial datum is the Dirac mass at the payoff kink $x = 1$ — nonnegative. Such equations preserve nonnegativity: probabilistically, $p(\tau, \cdot)$ is the density of the diffusion $dX_t = \sqrt{\nu(t, X_t)} X_t dW_t$ started at 1 (equivalently, apply the parabolic maximum principle to the adjoint). With $p \geq 0$ established, (ii) is immediate from (2) itself: $\partial_\tau c = \frac{1}{2} \nu x^2 p \geq 0$. $\square$

Pause on what has just happened, because it is the entire strategy of the model. Positivity of *one* function of two variables — the input ν — propagates *both* static no-arbitrage families

across the whole surface, for free, forever. The rest of this note is the discipline of not squandering that theorem: choose a parametrization in which $\nu > 0$ is trivial to enforce (section 3), choose a discretization that inherits the maximum principle rather than merely approximating it (section 5), and then *measure* what little the finite grid leaks instead of assuming the theorem carried over (section 5.3).

3 A surface you can hold

3.1 Finitely many positive numbers

The unknown is now a positive function of two variables — an infinite-dimensional object with a one-sided constraint, which sounds no easier than where we started. The move that makes it finite is the standard one from finite elements, and its payoff here is bigger than convenience.

Definition 1 (P_1 local-variance surface). On a tensor grid of $n_t \times n_x$ vertices (τ_i, x_j) the local variance is the continuous piecewise-affine interpolant

$$\nu_\theta(\tau, x) = \sum_{\ell=1}^m \theta_\ell \phi_\ell(\tau, x), \quad m = n_t n_x, \quad (3)$$

where the hat functions ϕ_ℓ are barycentric on a (cached Delaunay) triangulation of the vertices — each grid rectangle split into two triangles, the interpolant genuinely affine on each — and the parameters $\theta_\ell = \nu(\tau_i, x_j) > 0$ are the *nodal local variances*, flattened time-major.

Figure 2 shows the object itself — the sheet as the application renders it — and fig. 3 takes it apart.

Exercise 1. The obvious alternative on a rectangular cell is *bilinear* interpolation. Show that the bilinear interpolant of corner values $(0, 1, 1, 0)$ on the unit cell contains the term xy — so it is not affine, and its restriction to a diagonal is quadratic. Convexity of the weights holds either way, so proposition 2 below would survive the bilinear choice; the affine one is preferred because a P_1 function is determined by its values on *any* triangle, not only on axis-aligned rectangles — which is exactly what will let the same machinery run on the non-tensor vertex sets of section 13.

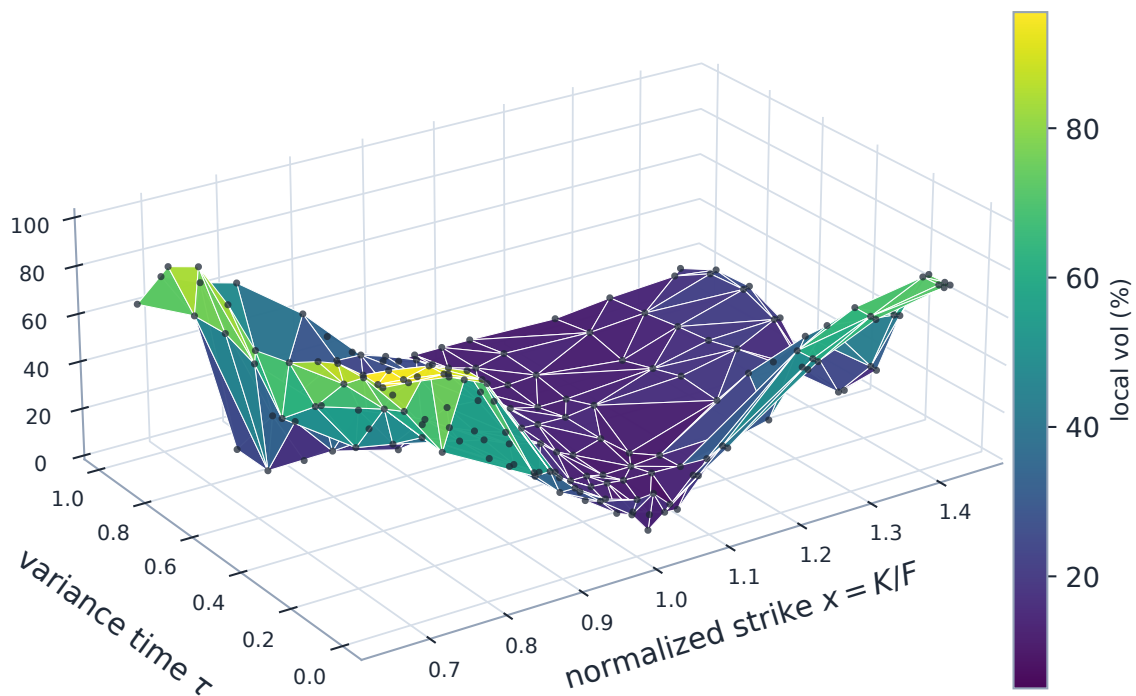


Figure 2: The model, drawn: a real SPY local-volatility sheet, fitted end to end (176 vertices), with the triangulation the pricing basis actually uses (remark 1 explains the mixed diagonals). Every triangle is a region where local variance is an affine function of (τ, x) ; every vertex (dot) is one calibration parameter; the put-side wall, the short-end ridge and the quiet call-side plain are all just nodal values. The whole surface is this list of positive numbers — nothing else.

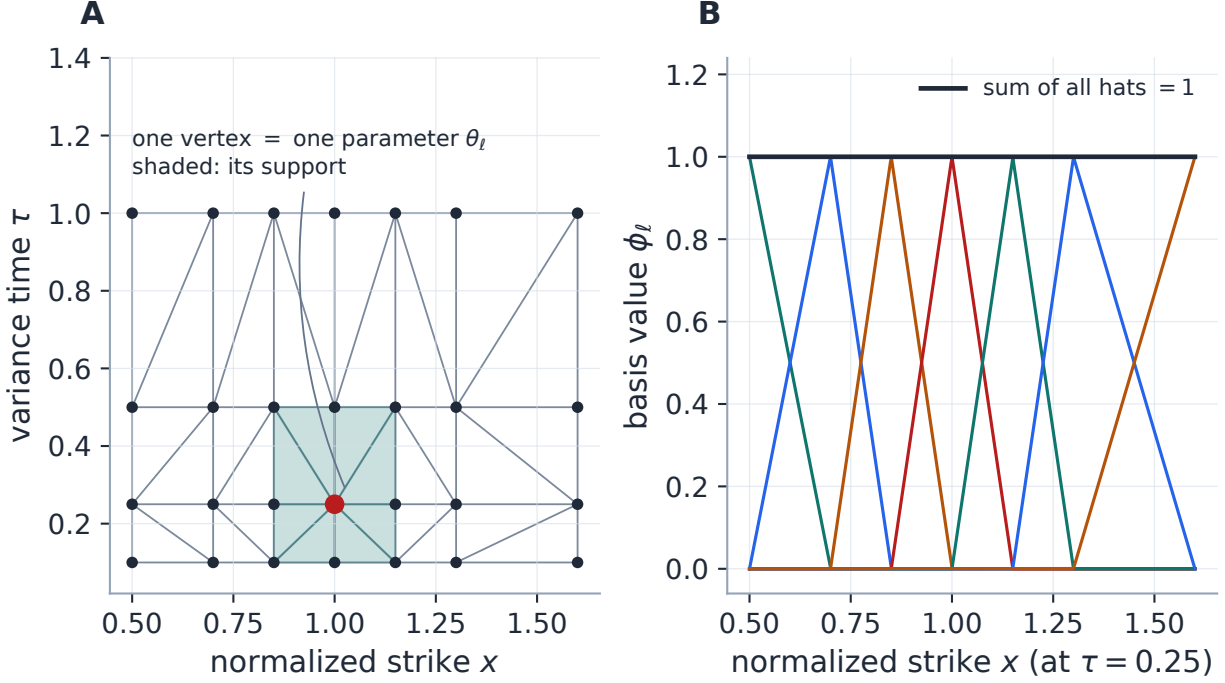


Figure 3: The anatomy of the sheet, from the production basis code. A: the triangulated vertex grid of the synthetic running example; the shaded star of triangles is the support of one vertex’s hat function — the entire region of the surface that its parameter θ_ℓ influences. Note the cell diagonals are *not* uniformly oriented; that is real, not a plotting artifact (remark 1). B: a strike cross-section of the hat functions at fixed τ : each is 1 at its own vertex, 0 at every other, affine between — and they sum to 1 identically (black line, the partition of unity that proposition 2 leans on; test-locked).

Remark 1 (A degenerate Delaunay, and which diagonal you get). The reader with sharp eyes will ask why the triangles’ hypotenuses in figs. 2 and 3 are not all parallel. The answer is a small lesson in computational geometry. On a tensor grid every cell is a rectangle, and a rectangle’s four corners are *cocircular* — the degenerate configuration in which *both* diagonals are equally Delaunay. The triangulation library therefore tie-breaks cell by cell (deterministically for given coordinates, but by no designed convention), and the pricing basis inherits that patchwork; the figures draw it faithfully. Does the choice matter? The two candidate interpolants of a cell agree at all four vertices and along all four edges — an edge restriction is linear either way — and differ only in the interior, by half the cell’s *twist* $(\theta_{i,j} + \theta_{i+1,j+1} - \theta_{i,j+1} - \theta_{i+1,j})/2$ at the centre: second-order small for smooth surfaces, and zero wherever the nodal values are locally bilinear. The application’s surface viewer originally split every cell along the same fixed diagonal for display — so the drawn sheet and the priced sheet disagreed by exactly this interior twist term; as of this revision the fit exports its per-cell diagonal orientation and the viewer consumes it, so what is drawn *is* the priced triangulation (test-locked). The residual honest caveat is the other direction: the priced interpolant’s diagonal orientation remains an artifact of the tie-break — stable because the triangulation is cached and regression-locked, but a convention-free degree of freedom that a hardened implementation would fix explicitly.

Why affine, and why triangles? Because affine interpolation is a *convex combination*: on each triangle, ν_θ is a weighted average of its three vertex values with nonnegative barycentric weights.

Averages cannot escape the range of what they average, which is the whole theorem:

Proposition 2 (Nodal positivity is surface positivity). *If $\theta_\ell \geq \nu_{lo} > 0$ for all ℓ , then $\nu_\theta(\tau, x) \geq \nu_{lo}$ everywhere on the triangulation’s hull; likewise every nodal upper bound is a surface upper bound.*

This is the two-dimensional counterpart of Note 01’s free chart, and it is why the calibration will need only *box bounds*: the positivity of a surface has been reduced to the positivity of finitely many numbers — a constraint every least-squares solver handles natively, with no penalty and no feasibility drama. The box itself is generous and mildly adaptive: a cap a fixed multiple of the largest implied volatility in view, and a floor a little *below* the smallest — why the floor must sit below the implied vols is a small fact about averages we prove in section 4. (Exact constants: appendix A.)

3.2 Beyond the hull: the wing contract

The grid must end somewhere, and pricing does not. Beyond the lowest strike vertex the surface continues local variance *linearly*, with slope a times the first interior cell’s slope; the right wing is flat-clamped. Three regimes for a : the default $a = 0$ is a flat clamp; a fixed multiple ($1.5\times$) can be requested to keep an unquoted put wing rising; and a becomes a *free* calibration variable, bounded $[0, 20]$, exactly when a variance-swap quote is present — the one instrument in the problem that genuinely carries information about the deep-put tail (section 6; its Jacobian column is analytic).

Caution: the extrapolation is outside the positivity guarantee

Proposition 2 stops at the triangulation hull, because outside it the basis carries *signed* weights — extrapolation is not an average. With $a > 0$ and a first cell whose variance decreases toward $x_{\min}$, the linear continuation can go *negative* before $x = 0$ (pricing floors it, so the priced curve kinks rather than blows up); symmetrically, a rising continuation exceeds the nodal cap freely — the cap is deliberately not applied in the extrapolation region (`models/localvol/affine.py`). Every guarantee in this note that leans on $\nu > 0$ or $\nu \leq \nu_{hi}$ is therefore scoped to the hull (or to the flat-wing $a = 0$ path, where the clamp inherits the boundary vertex’s box); an extrapolation-domain positivity constraint is an open follow-up.

Remark 2 (Lee’s bound, on the hull). Bounded positive local variance bounds implied total variance uniformly in strike — by comparison with the constant-variance diffusion, $w(k, \tau) \leq \nu_{hi} \tau$ — so the implied wing slope decays to zero, strictly inside Lee’s model-free cap of 2 (Notes 01 and 09). Scope it like everything above: the comparison holds where $\nu \leq \nu_{hi}$ holds, i.e. on the hull and the flat-clamped wings; a steep free- a put continuation escapes the cap and with it this bound. The affine surface carries no wing penalty and reports no per-fit Lee slopes — wing behaviour is a stated extrapolation contract plus the measured grid diagnostics, not a reported slope.

4 What the grids must resolve

Two grids inhabit this model and should not be confused: the *vertex grid* that carries the parameters, and the (much finer) *PDE lattice* that carries the march. Where to put both is not a matter of taste; each placement rule below is forced by a length scale or an identifiability fact of the problem, and each was learned the hard way (section 10).

The natural length scale. Everything interesting about a smile at maturity τ happens within a few multiples of the diffusive width $\sigma\sqrt{\tau}$ of the money. This single quantity dictates three choices at once. *Vertex placement:* strike vertices are spaced by *delta* — equal steps in Φ^{-1} -probability rather than in strike — so vertex density automatically follows where optionality lives. *Vertex coverage:* a vertex axis sized to the *longest* expiry undersamples a short one, whose whole smile can fall between two vertices — a two-day smile at 20% vol is $\sim 1.5\%$ wide, an order of magnitude narrower than a one-year smile — so every expiry must be guaranteed enough vertices *within its own* $\sigma\sqrt{\tau}$ -range, spread evenly across it (a count alone can be met entirely on one side). *Lattice steps:* the same width bounds the strike step, $\Delta x \lesssim 0.15 \sigma\sqrt{\tau_{\min}}$, and the number of time steps a short interval receives — a piecewise-affine surface cannot be priced more accurately than the lattice resolves its shortest smile, and if the *quote* spacing is finer than Δx , the fitted smile will oscillate at quote frequency until the lattice out-resolves it.

An averaging fact, and the floor. Here is the small theorem promised in section 3. At the money, implied variance is (approximately, and exactly in the time-homogeneous-at-the-money limit) a *time average* of local variance along the money path: $\sigma_{\text{ATM}}^2(\tau) \approx \frac{1}{\tau} \int_0^\tau \nu(s, 1) ds$. An average can only be matched by values that lie on both sides of it: whenever the ATM term structure is *increasing*, the early local variance must sit *below* the shortest implied variance. A parameter floor set carelessly at, say, 5% vol therefore makes some low-vol short-dated term structures literally unfittable — the optimizer rides the box and the misfit looks like a mystery. Hence the floor adapts to a fraction of the smallest ATM implied vol in view, and is keyed to ATM quotes deliberately: a noisy deep-wing quote must not be allowed to lower the floor where the box does real stabilizing work.

Identifiability below the first expiry. The quotes at the first expiry τ_1 pin — through the averaging fact above — only the *integral* of local variance over $[0, \tau_1]$. If several vertex rows lie inside that interval, their individual values are unidentified: the data constrains their sum, and the optimizer can ring one row up and the next down at no cost in fit (5–30 vol points of measured ringing, in practice). The cure is not more data but an honest prior: tie every sub-front row to the first identified row (the *front tie* of section 7), turning an unidentified subspace into a deliberately-chosen constant continuation. This is section 7.1’s theme in miniature, met before its section.

All of these refinements are *gated* on the regime that needs them — an ordinary surface takes none of these branches — and their exact thresholds and constants are catalogued in appendix A.

5 Pricing on the lattice

5.1 The discrete march

The forward map (2) is solved on a strike grid over $[0, x_{\max}]$, $x_{\max} \gtrsim 2.5$, default step $\Delta x = 0.01$ (refined for short-dated surfaces per section 4), with Dirichlet boundaries $c(\cdot, 0) = 1$ and $c(\cdot, x_{\max}) = 0$, by *fully implicit* Euler steps of size $\Delta\tau = 0.01$, the local variance entering at the new time level. On a possibly non-uniform grid with gaps $h_i^- = x_i - x_{i-1}$ and $h_i^+ = x_{i+1} - x_i$, the operator A encoding

$\frac{1}{2}\nu x^2\partial_{xx}$ has the three-point stencil

$$A_{i,i\mp 1} = \nu_i a_i^\mp, \quad A_{ii} = -\nu_i (a_i^- + a_i^+), \quad a_i^\mp = \frac{x_i^2}{(h_i^- + h_i^+) h_i^\mp} > 0, \quad (4)$$

and each step solves one tridiagonal system,

$$B^{n+1}U^{n+1} = U^n + (\text{boundary terms}), \quad B^{n+1} = I - \Delta\tau A^{n+1}. \quad (5)$$

Now the promised discipline: does this discretization *inherit* proposition 1, or merely approximate it? The answer runs through one classical linear-algebra fact, and it is worth seeing exactly how little is needed.

Proposition 3 (The implicit step matrix is an M -matrix). *For any $\nu \geq 0$ and any $\Delta\tau > 0$, the matrix $B = I - \Delta\tau A$ of (5) has positive diagonal $B_{ii} = 1 + \Delta\tau \nu_i (a_i^- + a_i^+) \geq 1$, non-positive off-diagonals $B_{i,i\mp 1} = -\Delta\tau \nu_i a_i^\mp \leq 0$, and unit row sums at interior rows. Hence B is a strictly diagonally dominant nonsingular M -matrix, and $B^{-1} \geq 0$ elementwise.*

Proof. Read the three properties off (4): the diagonal exceeds the absolute off-diagonal sum by exactly the 1 of the identity, $B_{ii} - |B_{i,i-1}| - |B_{i,i+1}| = 1 > 0$, and the sign pattern (positive diagonal, non-positive off-diagonals, dominance) is the defining pattern of a nonsingular M -matrix; M -matrices are inverse-positive [5] (concretely: Jacobi iteration for $Bu = v$ with $v \geq 0$ keeps nonnegative iterates and converges by dominance). $\square$

$B^{-1} \geq 0$ is the discrete ghost of the heat kernel being positive, and everything follows from it:

Corollary 1 (Discrete maximum principle, unconditionally). *For any $\Delta\tau > 0$ the scheme (5) is monotone ($U^n \geq V^n$ with the same boundary data implies $U^{n+1} \geq V^{n+1}$) and satisfies*

$$\min(\min_i U_i^n, c_{\text{bdry}}) \leq U_i^{n+1} \leq \max(\max_i U_i^n, c_{\text{bdry}}):$$

the march is unconditionally ℓ^∞ -stable, with no CFL restriction, and cannot create new extrema.

Proof. Monotonicity is $B^{-1} \geq 0$ applied to $U^n - V^n \geq 0$. For the bounds, let M be the larger of $\max_i U_i^n$ and c_{bdry} : unit row sums give $B(M\mathbf{1}) \geq M\mathbf{1}$ componentwise, $M\mathbf{1}$ dominates the right-hand side of (5), so $U^{n+1} \leq M\mathbf{1}$ by inverse positivity; the lower bound is symmetric. $\square$

Remark 3 (What the M -matrix argument does and does not prove). Proposition 3 and corollary 1 control the call *values*: order preservation, boundedness, no invented extrema. They do *not* by themselves give nonnegative second differences — a bounded monotone call vector can still be locally concave — so “the scheme cannot manufacture negative densities” would need a separate convexity-preservation proof for the actual stencil and boundary elimination (a discrete Fokker–Planck argument) that this note does not carry. The honest statement is the one production implements: the scheme is monotone and stable, and butterfly and calendar cleanliness are *measured* on the output grid, per fit (section 5.3); the arbitrage-cleanliness tests in the suite are regression checks on selected surfaces, not a general proof.

Strict dominance also means no pivoting is ever needed: each step is a plain Thomas march — the kernel the compiled march of section 9 vectorizes. The scalar algorithm fits in a listing:

Listing 1: The no-pivot tridiagonal (Thomas) solve for one implicit step (5) — the scalar algorithm that `models/localvol/affine_march.py` vectorizes; matches a dense solve to 10^{-10} .

```

1  def thomas(a, b, c, d):          # a: sub-, b: main, c: super-diagonal
2      n = len(d); cp = np.empty(n); dp = np.empty(n)
3      cp[0], dp[0] = c[0]/b[0], d[0]/b[0]
4      for i in range(1, n):        # forward sweep (no pivoting: Prop. M-
                                   matrix)
5          m = b[i] - a[i]*cp[i-1]
6          cp[i], dp[i] = c[i]/m, (d[i] - a[i]*dp[i-1])/m
7      x = np.empty(n); x[-1] = dp[-1]
8      for i in range(n-2, -1, -1):
9          x[i] = dp[i] - cp[i]*x[i+1]
10     return x

```

5.2 Why not Crank–Nicolson

First-order time accuracy looks like a price worth negotiating, and Crank–Nicolson — replace (5) by $(I - \frac{\Delta\tau}{2}A)U^{n+1} = (I + \frac{\Delta\tau}{2}A)U^n$ — is the standard second-order offer. Look at what it costs. The implicit factor is still an M -matrix, but the *explicit* factor has diagonal $1 - \frac{\Delta\tau}{2}\nu_i(a_i^- + a_i^+)$, nonnegative only under the CFL-like bound $\Delta\tau \leq 2/(\nu_i(a_i^- + a_i^+))$ — violated precisely on coarse strike grids with high local variance, which is where realistic lattices economize. When it is violated the scheme remains second-order *accurate* but is no longer *monotone*, and the payoff kink excites the classic CN oscillation.

Exercise 2. On a uniform grid the bound reads $\Delta\tau \leq 2\Delta x^2/(\nu x^2)$. Evaluate it at the money for a 40%-vol surface ($\nu = 0.16$) on a $\Delta x = 0.025$ lattice: $\Delta\tau \leq 7.8 \times 10^{-3}$ — already below the standard $\Delta\tau = 0.01$ marching step, before any refinement of the strike grid makes it quadratically worse. Monotone CN on realistic lattices is not an available option; damped start-up steps are the standard repair [4], and fig. 4 shows what they are repairing.

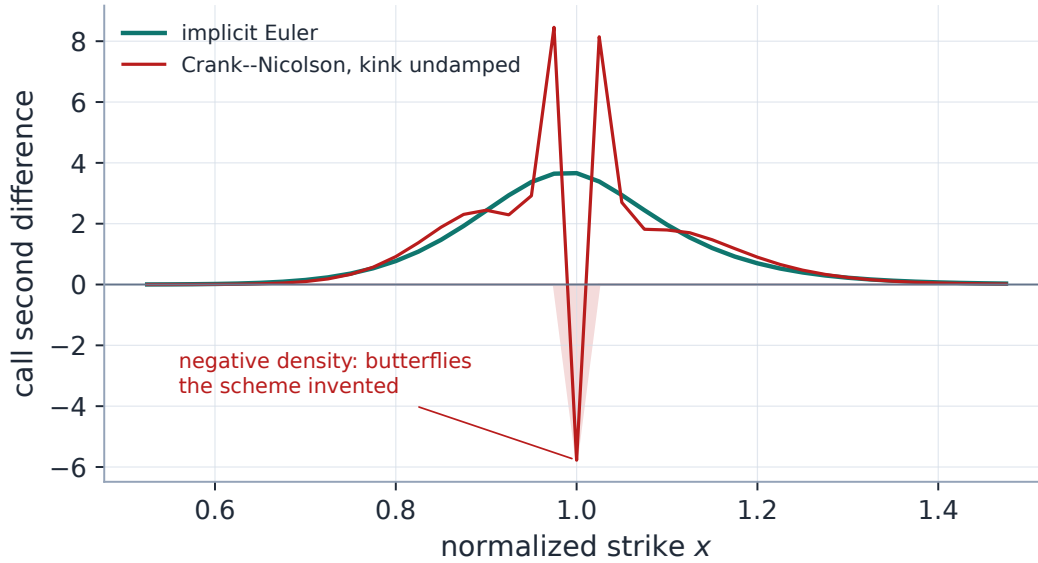


Figure 4: Monotonicity is a property of the scheme, not of luck. The same 40%-vol surface, the same lattice, marched to $\tau = 0.1$ two ways by the same solver. Implicit Euler (teal): a clean bell of second differences, minimum 2.60×10^{-3} — corollary 1 at work. Crank–Nicolson reaching the payoff kink essentially undamped (rust; the implicit start-up step was made vanishingly short): the second difference swings to -5.77 — a large *negative density*, butterflies the scheme invented out of nothing. This is why the default scheme trades one order of time accuracy for the maximum principle, and why the CN variant exists only behind Rannacher start-up steps that damp the kink first.

Nor is the trade-off hypothetical: when the Rannacher/CN variant was evaluated as a speed lever it bought little and still produced a measured coarse-grid arbitrage violation on a test surface, so the default remains fully implicit (appendix B).

5.3 Reading out, and measuring what leaks

The observation operator reads the normalized call at each quote’s (τ, x) off the marched solution by interpolation. Then the implementation does what remark 3 demands and *measures* what the discretization leaks, with explicit tolerances: the per-expiry minimum second difference (butterfly proxy) must be $\geq -10^{-6}$; adjacent maturities may not decrease by more than 10^{-9} (calendar); prices must sit in $[-10^{-9}, 1 + 10^{-9}]$ — all three reported with every fit.

Two further layers deserve a lecture’s emphasis, because both are about *where numerical error hides in an inverse problem*. First, fit quality is re-priced on a refined operator $(\Delta\tau/4, \Delta x/2)$. The reason is subtle and general: an optimizer facing a fixed discretization will happily bend its parameters to cancel the operator’s *own* error, so the in-operator residual systematically flatters the fit; only the refined reprice estimates the distance to the true prices. (In the inverse-problems literature this is a cousin of the “inverse crime” — validating a reconstruction with the same operator that produced it.) The exposure is certification-locked: a deliberately coarsened march *must* be revealed by the converged metric (`rmsConvergedBp`, reported beside `arbitrageFree` with every fit; case `lv_operator_blindness`). Second, an independent discretization — a finer log-strike Crank–Nicolson/Rannacher scheme — cross-checks the march: two different solvers agreeing is evidence; one solver agreeing with itself is not.

6 Variance swaps

A variance-swap quote is the fair strike of realized variance — for a diffusion, the expected integrated local variance to expiry. Two objects share the section: the *total* $I(\tau) = \mathbb{E} \int_0^\tau \nu(s, X_s) ds$, which the residual block prices, and the quoted annualized fair strike $I(\tau)/\tau$. Production carries two pricers:

- **Static replication** (default): the log-contract weights

$$I(\tau) = 2 \int_0^1 \frac{P(\tau, x)}{x^2} dx + 2 \int_1^\infty \frac{C(\tau, x)}{x^2} dx, \quad (6)$$

integrated by trapezoid over the PDE strike grid itself — no separate replication grid — with the put leg starting at the fixed normalized strike $x = 0.01$ (the x^{-2} weight diverges at zero), not at the lowest quoted strike.

- **Source PDE** (the alternative): solve one *backward* equation $\partial_t g + \frac{1}{2} \nu x^2 \partial_{xx} g + \nu = 0$, $g(\tau, \cdot) = 0$, and read $I(\tau) = g(0, 1)$ — Feynman–Kac for the running integral, on the same tridiagonal machinery marched backward, with *analytic* sensitivities $\partial I / \partial \theta$ (and $\partial I / \partial a$ for the free left slope) through the same multi-RHS trick. Its selling point, stated carefully: $g(0, 1)$ is determined by the diffusion *around* $x = 1$, so it is materially less sensitive to the far-strike boundary than the x^{-2} -weighted replication tail — but it still runs on the finite domain with approximate boundary conditions, so “no truncation tail” would be too absolute. Note 08 develops both in full.

7 What the data can see, and the objective

7.1 Two surfaces, one set of quotes

Before writing an objective, a lecture should ask whether the problem it is about to pose has one answer. This one does not, and it is better to see that than to be told it. A typical fit has $m \approx 100$ – 200 nodal parameters (plus the free left slope when a var-swap is quoted) against a few dozen vanillas concentrated near the money of a handful of expiries — and even where the counts balance, a vanilla constrains local variance only through a smoothing integral along diffusion paths. Whole regions — deep wings, sub-front rows, times beyond the last expiry — are weakly determined. Figure 5 makes the point concrete: two surfaces that differ by 22 vol *points* in the deep-put column reprice the same quote set to within 1.6 vol bp RMS (baseline: 0.2). A thousand basis points of surface, two of quotes.

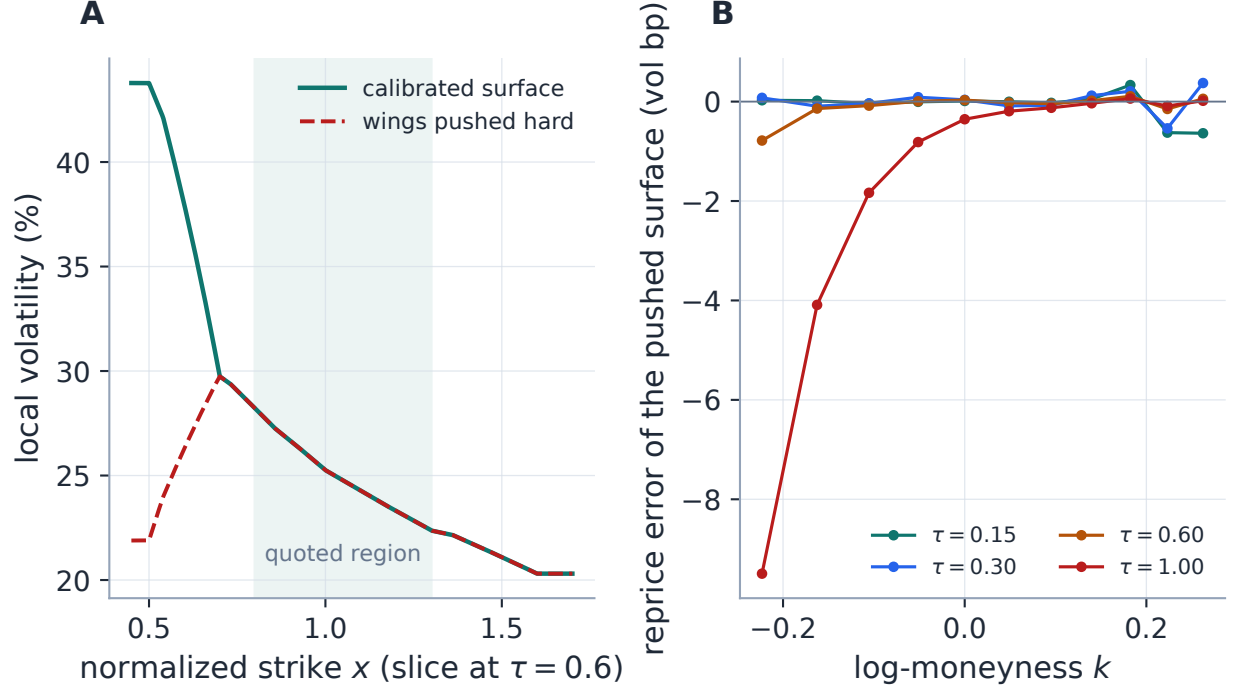


Figure 5: What sparse vanillas do not see. A: the calibrated synthetic surface (teal) and the same surface with its unquoted deep-put column ($x = 0.5$, well below the lowest 0.80 strike) pushed down by 22 vol points (dashed); the shaded band is the quoted region, where the two are indistinguishable. B: the pushed surface’s reprice errors at every quote: at most a few vol bp, concentrated in the longest expiry’s lowest strikes — the only quotes whose diffusion cone reaches the moved column at all. Low quote error is *not* surface recovery; it is one representative of an equivalence class.

The consequence is philosophical and operational at once: what the calibration returns is one *regularized representative* of that equivalence class, and every regularizer below is a declared choice of representative. The roughness block selects smoothness toward θ_{ref} ; the box clips the unidentified extremes; the front tie pins sub-front rows to the first identified row; the prior baskets pull toward yesterday where quotes are silent; and the seed decides which basin a non-convex solve lands in. The worked example of section 11 therefore reports quote error and surface error as two separate numbers.

7.2 The objective

With unweighted residual blocks $r_{\bullet}$, the fit minimizes a stacked, bound-constrained least squares over the decision vector — θ alone on the hot path, or (θ, a) with $a \in [0, 20]$ when the left slope is fitted:

$$\min_{\substack{\theta \in [\nu_{\text{lo}}, \nu_{\text{hi}}]^m \\ (a \in [0, 20])}} \frac{1}{2} \left(\|r_{\text{opt}}\|^2 + \|r_{\text{vs}}\|^2 + \|r_{\text{bsk}}\|^2 + \lambda \|L(\theta - \theta_{\text{ref}})\|^2 + w_{\text{cvx}} \|r_{\text{cvx}}\|^2 + w_{\text{ft}} \|r_{\text{ft}}\|^2 \right), \quad (7)$$

each weight appearing once: production stacks the rows $\sqrt{w_{\bullet}} r_{\bullet}$, whose squared norm is (7) exactly. The blocks, each with its role in the representative-choosing story:

- **Option residuals** $r_{\text{opt},q} = (P_q(\theta) - y_q)/\eta_q$ in vega-normalized forward-call units — the same price-space-solve, vol-space-error device as Note 01 — with η_q a floored vega scale; or a bid–ask band hinge with a small mid anchor when band edges are present (Note 07).
- **Var-swap residuals** $r_{\text{vs}} = (I(\theta) - z)/\zeta$ in total variance, one scalar per quoted expiry (Note 08). It constrains the *global* tail integral (6), biasing deep-put mass in aggregate — it does not identify the tail’s shape.
- **Operator baskets** r_{bsk} : the prior-persistence operators (Note 13) enter as *signed baskets* of option prices — one residual per ATM/RR/BF operator, preserving the risk-reversal and butterfly coupling instead of scattering it into synthetic quotes.
- **Roughness** $L(\theta - \theta_{\text{ref}})$ at weight λ : Tikhonov regularization with a curvature seminorm — L is the spacing-aware second difference (cell-width-normalized divided differences, reducing to $(1, -2, 1)$ on a uniform grid, exact curvature on a non-uniform one), with time-vs-strike balance ρ . This is the smoothness that lets a coarse grid generalize — and the declared answer to section 7.1: of the equivalence class the data allows, take the smoothest member near the reference.
- **Convex wing** (optional) at weight w_{cvx} : a one-sided hinge $\text{relu}(-D^2\sigma)$ on the nodal local *vols*, confined to vertices at or below the 5Δ put *and strictly below the deepest observed quote* — the extrapolation tail only. Why the confinement is worded so precisely is the second case file of section 10.
- **Front tie** (on by default) at weight w_{ft} : the one-sided differences $\theta_{i,:} - \theta_{i+1,:}$, tying the unidentified sub-front rows to the first data-identified expiry row — the single $\tau = 0$ row normally, the full chain at weight ≥ 1 on short fronts (section 4).

8 Every derivative from the same factorization

A least-squares solver lives on Jacobians, and a PDE-implicit pricing map sounds like finite-difference territory — m rebuilds of the march per iteration. It is not, and the reason is the same structural fact that served Note 01: the map from parameters to the operator is *linear*. Since ν_θ is linear in θ (3), so is $A(\theta)$, with $\partial A/\partial\theta_\ell = A[\phi_\ell]$ — the stencil (4) with ν replaced by the hat function ϕ_ℓ .

Proposition 4 (Tangent: same factor, extra right-hand sides). *The sensitivity $s_\ell^n = \partial U^n/\partial\theta_\ell$ obeys*

$$B^{n+1}s_\ell^{n+1} = s_\ell^n + \Delta\tau A[\phi_\ell]U^{n+1}, \quad s_\ell^0 = 0, \quad (8)$$

the discretization of the continuous sensitivity PDE $\partial_\tau s_\ell = \frac{1}{2}\nu x^2 \partial_{xx} s_\ell + \frac{1}{2}\phi_\ell x^2 \partial_{xx} c$.

Proof. Differentiate $B^{n+1}(\theta)U^{n+1}(\theta) = U^n(\theta)$ in θ_ℓ and use $\partial_{\theta_\ell} B^{n+1} = -\Delta\tau A[\phi_\ell]$, which is exact by linearity. (Audited against central differences in fig. 6.) $\square$

Read (8) the way a performance engineer would: every sensitivity marches behind the *same* tridiagonal factor as the value solve, so the entire m -column Jacobian costs one factorization plus m extra back-substitutions per step — $O(N_\tau N_x m)$ — and the m columns form a contiguous inner loop that vectorizes beautifully (section 9). Read it the way a probabilist would and it is just as pleasant: the source term says that vertex ℓ influences a price exactly through the diffusion’s visits to ℓ ’s support, weighted by the convexity it finds there. Figure 6 shows both readings.

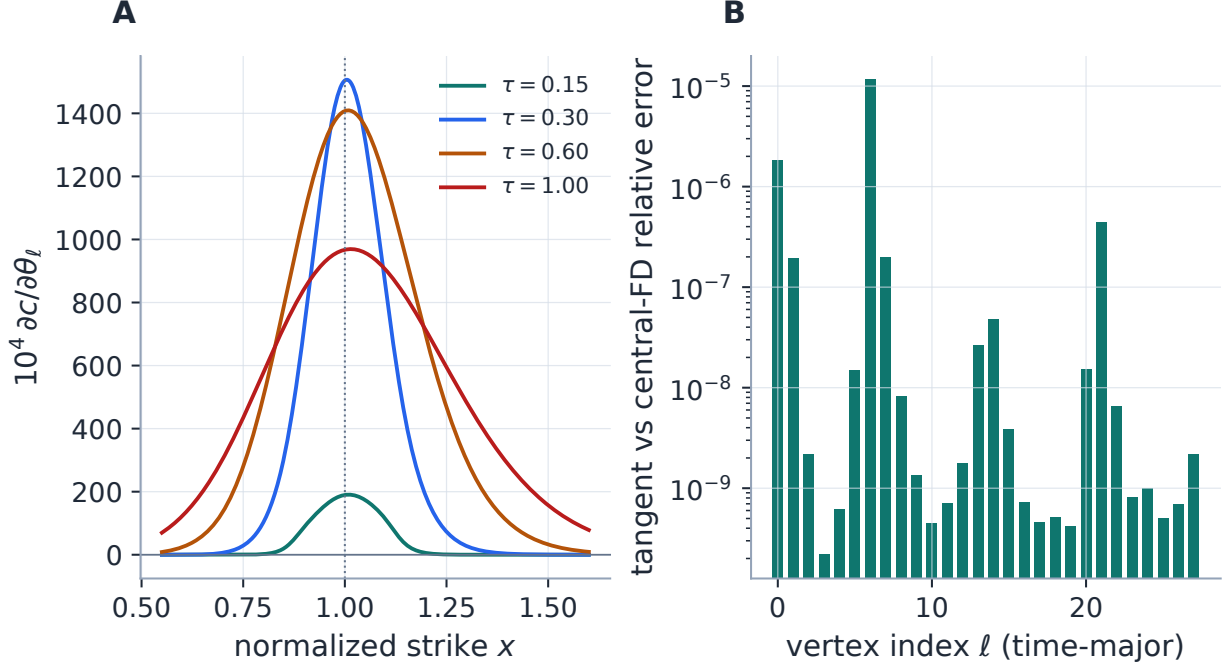


Figure 6: The tangent system, seen and audited. A: the sensitivity of the priced call curve to *one* interior vertex ($x = 1.0$, $\tau = 0.25$; dotted line marks its strike) across the four quoted expiries: the influence is a cone — widest near the vertex’s own time-and-strike neighbourhood, spreading and flattening with maturity as the diffusion forgets where it collected its variance. B: the audit: each vertex’s analytic column against a central finite difference of the full march — worst relative disagreement 1.16×10^{-5} across all 28 vertices.

8.1 The adjoint, derived and not yet needed

The tangent route costs $O(N_\tau N_x m)$. The classical alternative makes gradient cost independent of m , and deriving it costs three lines, so the note carries it — with its implementation status stated plainly.

Proposition 5 (Adjoint: one backward sweep). *Let $J(\theta) = f(HU^{1..N}(\theta))$ be any scalar objective reading the marched solution through observation rows H . Define adjoint states by the backward recursion*

$$(B^n)^\top \mu^n = H_n^\top \partial f_n + \mu^{n+1}, \quad \mu^{N+1} = 0. \quad (9)$$

Then

$$\frac{\partial J}{\partial \theta_\ell} = \sum_n (\mu^n)^\top \Delta \tau A[\phi_\ell] U^n, \quad (10)$$

at the cost of one backward PDE solve, $O(N_\tau N_x)$, plus the $O(m)$ output accumulation — the PDE cost independent of m .

Proof. Form the Lagrangian $\mathcal{L} = f(HU) - \sum_n (\mu^n)^\top (B^n U^n - U^{n-1})$ (boundary terms suppressed). Stationarity in each U^n gives (9); the θ -derivative of $\mathcal{L}$ then picks up only the explicit dependence of B^n , which is $-\Delta \tau A[\phi_\ell]$, giving (10). This is summation by parts: the transpose of a lower

block-bidiagonal system is upper block-bidiagonal — a backward march with transposed tridiagonal factors. $\square$

What the implementation actually runs is narrower, and the distinction matters. It marches all forward sensitivities (8), materializes the dense option Jacobian, and evaluates both products as matrix products: `apply_jacobian` is Jv and the transpose product is literally $J^\top w$ on the assembled matrix (plus a sparse regularization block) — *not* the backward recursion (9). The matrix-free Gauss–Newton solver’s win is therefore purely in the linear algebra: it avoids forming $J^\top J$ and the dense per-iteration SVD, but its data Jacobian is still assembled from forward sensitivities at $O(N_\tau N_x m)$ per evaluation. The true PDE adjoint — gradient cost independent of m — is future theory, waiting for the non-tensor grids of section 13 where m finally justifies it. What is test-locked is the implemented pair: the product identity $\langle Jv, w \rangle = \langle v, J^\top w \rangle$ and the gradient against finite differences.

9 The cost of a fit

A lecture on a numerical method owes its audience the arithmetic of what it costs. One evaluation of the objective (7) and its Jacobian is a value march plus m tangent columns, $O(N_\tau N_x m)$; a trust-region least-squares iteration then pays the linear algebra of a dense $n_q \times m$ Jacobian — an SVD or factorization at $O(n_q m^2)$ — and a fit is a few dozen to a couple of hundred such evaluations. At realistic sizes ($m \sim 100$ – 500 , a few hundred quotes, lattices of 10^2 – 10^3 nodes) no single term dominates: profiled cold fits split their time between the optimizer’s linear algebra, the sensitivity march, and Jacobian assembly, with the value solve itself almost free. Amdahl’s law then makes a prediction that was borne out repeatedly: any lever that accelerates *one* component — a coarser grid, a compiled kernel in the wrong loop order, a higher-order time scheme, a cleverer decomposition — buys a factor of 1.1 and no more. The levers that worked either cut the number of evaluations or moved the whole per-evaluation stack at once:

- **Early stopping as regularization-aware termination.** The data misfit converges long before the optimizer does: the tail iterations grind trade-offs between regularization terms far below quote noise. Tracking the option-block misfit and stopping at its best stalled iterate cuts evaluations — the one denominator every cost term shares — at a quantified, deliberately accepted sub-vol-bp distance from the full optimum.
- **The multi-right-hand-side structure.** Proposition 4 says all m tangent columns share one tridiagonal factorization per step; the computational corollary is that the m -column axis should be the *contiguous inner loop* of the solve, so the back-substitutions vectorize. Compiling the same algorithm with the loop order inverted gains almost nothing — memory layout, not compilation, is the lever.
- **Avoiding the normal equations.** A classical Gauss–Newton step solves $\min_\delta \|J\delta + r\|$; forming $J^\top J$ squares the condition number, and a dense decomposition of J costs $O(n_q m^2)$ per iteration. A matrix-free least-squares iteration (LSMR) needs only the products Jv and $J^\top w$ of section 8 and avoids both. It is scoped to where its assumptions hold: the smooth mid-fit objective — hinge (band) objectives break the smoothness Gauss–Newton linearizes, and those fits keep the trust-region solver, which also serves as the automatic fallback. The accepted trade is

stated openly: on real data the two solvers can converge to slightly different optima of the same non-convex problem, a fraction of a vol bp apart.

- **Warm starts as continuation.** Recalibrating a live surface is a continuation problem in data space: yesterday’s (or one second ago’s) surface is a starting point already inside the right basin. The reference θ_{ref} is held fixed so the *objective* is unchanged — warm starting changes the path, not the problem — though for a non-convex bound-constrained solve that is an empirical comfort, not a uniqueness theorem.

Together these turned an ~ 86 -second cold fit into a several-times-faster one with near-instant recalibrations; the measured multipliers, machine scopes and the ledger of shelved levers are collected in appendix B, where machine-dependent numbers belong. One piece of history stays in the body because its lesson is methodological:

The same algorithm, wrong twice, right once

The matrix-free Gauss–Newton now serving as the default was first evaluated before the compiled march existed — and lost: removing the dense decomposition made fits *slower*, because the bottleneck was then the march, not the linear algebra. Worse, a clean synthetic benchmark (zero residual, interior optimum) hid the loss — Gauss–Newton converged in 8 evaluations there while blowing past the evaluation cap on stiff real chains. Once the compiled march collapsed the per-evaluation cost, the linear-algebra share dominated and the *same* algorithm won. Two durable lessons for any numerical program: profile before optimizing, and never accept a synthetic-only benchmark — the same discipline later caught the constraint regression of section 10, which no synthetic test could see.

10 Case files: two lessons in numerical pathology

Numerical analysis, like trading, is mostly learned from losses. Two incidents from this model’s history each carry a lesson more general than the bug — one about resolution, one about constraints — and both reward retelling in full: setup, failure, diagnosis, fix, verdict.

Case file: the six-day weekly that broke the strike grid

Setup. A true six-day SPY weekly, from a captured live chain (the standing benchmark’s shortest expiry was three weeks — which is why the bug had survived it). Normal expiries fit at a few vol bp.

Failure. The weekly fit was catastrophic: 108 vol bp RMS against a parametric slice model’s ~ 47 on the same quotes — the surface visibly missing the smile’s curvature.

Diagnosis — measure first. A pure side-channel diagnostic counts, per expiry, the candidate causes: in-range strike vertices, vega-floored quotes, PDE time steps, active prior rows. The counts acquitted the usual suspects — time steps adequate, prior inactive, vega floor latent — and convicted the strike axis: sized to the *longest* expiry and clipped to the global range, it landed exactly 3 vertices on the weekly’s sharpest curvature (fig. 7).

Fix. Two gated changes. (i) A per-expiry coverage floor: split the widest in-range gaps until every expiry owns ≥ 8 vertices inside its own traded range — densifying only under-covered short fronts. (ii) A short-expiry-aware PDE step: refine the shared Δx to a fraction of the smallest ATM $\sigma\sqrt{\tau}$, snapped so $x = 1$ stays a node ($0.3\times/400$ nodes at the time; since tightened

to $0.15 \times /800$ as section 4's resolution rules hardened).

Verdict. $108 \rightarrow \sim 28$ vol bp from the coverage floor alone, 23.5 with the refined step — now *better* than the parametric fit — while every long expiry and every normal surface is unchanged to the byte (both gates test-locked, and the incident is registered in the named certification pack as `weekly_lv_resolution`). The general lesson is section 4's sampling principle wearing production clothes: *a lattice that does not resolve the data's length scale fits an alias, not the data* — and the way to find such a bug is to count, not to guess.

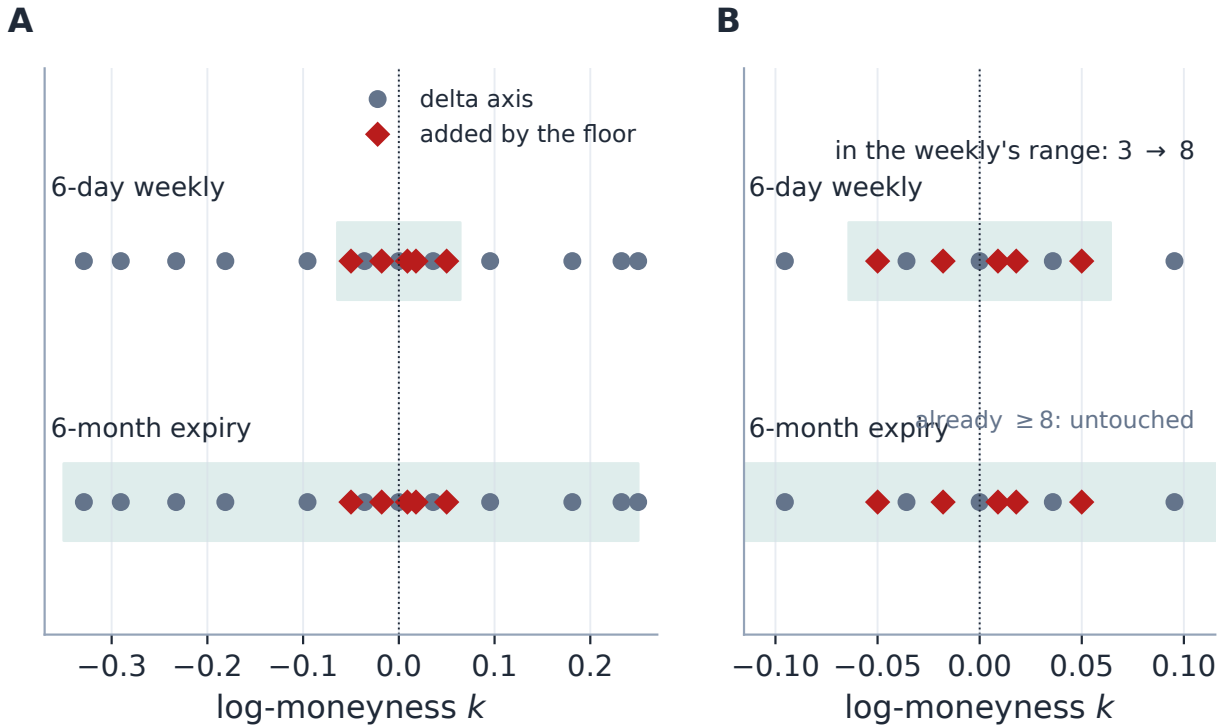


Figure 7: The rescue, computed by the production grid builder on a 6-day-plus-6-month universe. A: the delta-spaced axis (grey dots) is sized by the six-month expiry, so the weekly's narrow traded range (upper shaded lane) catches only 3 of its vertices; the coverage floor splits the weekly's widest in-range gaps (rust diamonds) until it owns ≥ 8 . B: zoomed to the weekly's range. The six-month lane already meets the floor, so not one split is made on its account — the gate is a no-op exactly where it should be.

Case file: the convex wing that flattened SPY

Setup. The convex-wing hinge of section 7 exists to keep the *unquoted* deep-put extrapolation convex. A user ran with a saved denser grid — 20 strike vertices against the default 12.

Failure. SPY's surface RMS degraded to 25.7 vol bp on the real benchmark; NVDA looked fine.

Diagnosis. The constraint originally selected every vertex at or below the 5Δ put, *regardless of data*. At 20 strike nodes several of those vertices sit inside SPY's densely quoted put wing, and the hinge fought the quotes — forcing the wrong wing shape on a low-vol name. NVDA's

naturally convex wing had hidden the bug, and so had the default grid: only the denser grid pushed constrained vertices into quoted territory.

Fix. Confine the hinge to vertices at or below the 5Δ put *and strictly below the deepest observed quote* — the extrapolation tail only.

Verdict. SPY $25.7 \rightarrow 2.6$ vol bp, NVDA unchanged; the confinement is test-locked and registered as certification case `convex_wing_tail`. The lesson echoes Notes 05 and 09: *a shape constraint must not be imposed where the data already speaks* — the same principle at the de-Am input, the MCS output, and here the LV extrapolation tail.

11 Worked examples

Provenance in one line: every number below is regenerated by the note’s figure generators at commit 77d43fb (2026-07-11), with configuration and library versions archived alongside (`figures/lv_numbers.json`).

11.1 The round trip, and its two separate numbers

A known skewed surface is priced through the production PDE, quoted at four expiries, and recovered from a *flat* seed through the low-level calibrator (TRF, banded march, custom dense grids: an algorithm check, not the product path or its timing). Per section 7.1 the result is two numbers, not one. The *quotes* reprice to a maximum of 0.6 vol bp in 33 evaluations (28 vertices). Separately, the *surface itself* agrees with the truth to 0.06 vol points RMS (0.25 max) on a dense grid over the quote-covered region — gratifyingly small here, because 44 clean quotes constrain 28 vertices of a smooth truth, but it is the measured surface error, not an inference from the quote error, and it degrades off the covered region and on sparse real chains (fig. 5 showed how far).

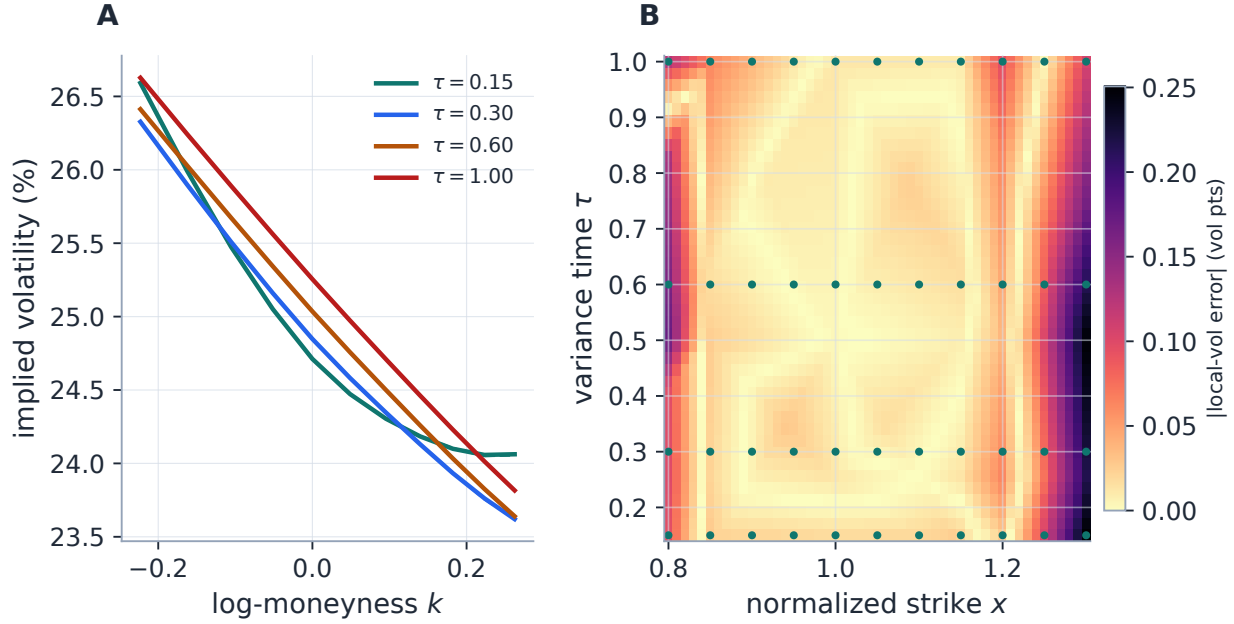


Figure 8: The synthetic round trip. A: target smiles (solid) and the recovered fit (dashed) at the four quoted expiries — the dashed curves are hidden under the solid ones at this scale (max quote error 0.6 vol bp). B: the honest second number: the local-vol error surface $|\text{recovered} - \text{truth}|$ over the quote-covered region, with the quote positions dotted; largest where quotes are farthest (between expiries, at the strike edges), 0.25 vol points at worst.

11.2 The real-chain benchmark

On a static snapshot of real SPY and NVDA chains, the full pipeline at shipped defaults reads, in two metrics per name (table 2, fig. 9): the in-operator surface RMS — 2.4 vol bp on SPY, 11.9 on NVDA — and the refined-operator reprice ($\Delta\tau/4$, $\Delta x/2$): 15.9 and 56.3. The gap between the two metrics is itself the finding: in-operator residuals are blind to time-discretization error the optimizer compensates, so the converged number is the honest fit quality, and closing the gap on short fronts (per-expiry $\Delta\tau$ refinement) is the open short-end follow-up. NVDA’s harder short expiries carry the larger residual in both metrics — the regime section 4’s resolution rules exist for.

Table 2: Full-pipeline local-vol fit on the real-chain snapshot (fresh run, commit 77d43fb). “Worst expiry” is in-operator.

Name	in-op RMS (bp)	converged RMS (bp)	worst expiry (bp)
SPY	2.4	15.9	3.4
NVDA	11.9	56.3	12.4

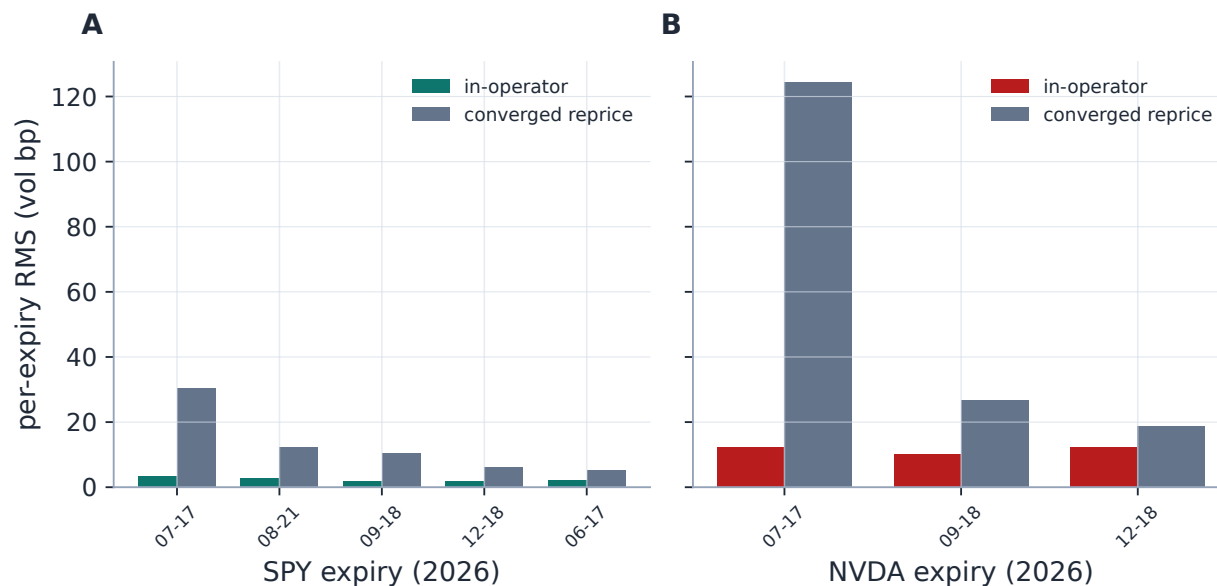


Figure 9: Per-expiry RMS on the real-chain snapshot, in-operator bars beside the refined-operator reprice, for SPY (A) and NVDA (B; note the shared scale). The residual lives at the short end in both metrics, and the converged bars are the honest ones — the very short NVDA front’s converged reprice dwarfs its in-operator figure, which is exactly the compensated-operator-error effect the second metric exists to expose.

12 What is genuinely original here

The P_1 -positivity idea and the forward-calibration philosophy are Andreasen–Huge [2]; the contributions of this implementation are depth and discipline:

1. the *vectorized-Thomas compiled march*, with loop order as the actual lever;
2. the *stall early-stop*, the one lever that scales march, assembly and optimizer together;
3. the *matrix-free Gauss–Newton* first rejected, then correctly promoted when the compiled march made the SVD dominant — consuming the forward-assembled Jacobian through the two matrix products of section 8;
4. the *source-PDE var swap* with analytic sensitivities;
5. the *measure-first* short-dated diagnosis that convicted the strike grid on counts rather than hunches.

Each was validated against a real benchmark that twice caught regressions a synthetic test would have missed.

One clarification of scope. This lecture’s object is the *direct fit* — the surface calibrated to quotes. A separate path *extracts* a local-variance grid from already-fitted parametric slices via the formula of appendix D (it feeds scenario transport and the cold seed); that is a different object with different error characteristics, and the two should not be conflated.

13 Limitations

Where the guarantees stop. A pure diffusion cannot make a true short-dated event spike (Note 11’s variance clock absorbs *scheduled* events instead). The tensor grid wastes vertices in the corners; the future non-tensor bowtie grid is where the GN no-SVD advantage — and the true adjoint of proposition 5 — will finally dominate. The extrapolation region sits outside the positivity guarantee (section 3). The converged-reprice gap on short fronts is open (section 11). And the fit, though much faster, remains the heaviest computation in the application.

A Hyperparameter atlas

Table 3: Local-volatility hyperparameters.

Knob	Default	Role
<i>Surfaced (OptionsSettings)</i>		
gridStrikeMode	delta	Strike-vertex placement (delta-spaced).
gridXNodes	12	Strike vertices of the variance surface.
gridXMinPerExpiry	8	Min in-range strike vertices per expiry (section 10).
gridTNodes	10	Maturity vertices.
gridRegLambda λ	10^{-2}	Roughness strength (model-layer signature default is 10^{-4} ; production passes this).
gridRegRho ρ	1.0	Time-vs-strike roughness balance.
convexWing	/ false / 10^3	Convex-wing hinge (extrapolation tail only).
convexWingWeight		
frontTie	/ true / 10^{-2}	Front-end tie of the $\tau = 0$ row to the first expiry row.
frontTieWeight		
lvVolCapMult	3.0	Adaptive variance cap $\min(\max(0.36, (\text{mult} \cdot \sigma_{\max})^2), 16)$ — at most 400% vol.
leftWingSlopeMult	1.5	Left local-variance extrapolation slope multiple.
varSwapMethod	static	Var-swap pricer: replication over the PDE grid, or source-PDE.
lvSolver	gn	Matrix-free Gauss–Newton (default) or trf.
lvFastKernel	true	Numba vectorized-Thomas march.
lvEarlyStop	true	Stall-based early stop.
timeScheme	implicit	Implicit Euler (monotone) or opt-in Rannacher/CN (section 5.2).
midAnchorWeight	0.05	Mid anchor inside the band objective.
<i>Hidden (affine PDE / grid / solver)</i>		
varLo / varHi	0.0025 / 0.36	Request-level nodal variance box (vol 5%/60%) before the adaptive cap and floor.
_LV_VOL_FLOOR_FRAC	0.5	Adaptive floor: $\nu_{lo} = \min(\text{request}, (0.5 \min \sigma_{\text{ATM}})^2)$ (section 4).
_X_DX / _X_MAX_MIN	0.01 / 2.5	PDE strike step and minimum $x_{\max}$.

Knob	Default	Role
<code>_DT_MAX</code>	0.01	Implicit-Euler maturity step (0.03 under Rannacher).
<code>rannacher_steps</code>	2	Implicit start-up steps before CN (opt-in scheme).
<code>_PDE_DX_SHORT_FRAC</code> / <code>_PDE_N_MAX</code>	0.15 / 800	Short-dated step: $0.15 \sigma \sqrt{\tau}$, capped at 800 nodes (was 0.3/400 before the daily-ladder pass).
<code>_PDE_NT_FIRST_GATE</code> / <code>_PDE_NT_SHORT</code>	8 / 32	Short-interval time refinement: 32 steps whenever the flat ceiling would give < 8 .
<code>_COVERAGE_GAP_MAX_T</code>	10/365	Expiry age below which the even-gap coverage rule applies (section 4).
<code>FRONT_TIE_SHORT_T</code> / <code>FRONT_TIE_CHAIN_WEIGHT</code>	0.08 / 1.0	Chained front tie: threshold and minimum effective weight.
<code>varswap_k_lo</code>	0.01	Fixed lower strike of the static replication’s put leg.
<code>_DELTA_SET</code>	7 deltas	Put deltas $\{1, 2, 5, 10, 25, 40, 50\}\%$ (and call mirrors) spacing the strike axis.
<code>_CONVEX_WING_DELTA</code>	0.05	Wing boundary for the convex hinge (5Δ put).
<code>_VOL_TOL</code> / <code>_VEGA_FLOOR</code>	$0.01 / 10^{-3}$	Vega normalization of option residuals.
diag tolerances	$-10^{-6} / 10^{-9}$	Butterfly proxy floor; calendar/bounds tolerances (section 5.3).
<code>x_scale</code> / tolerances	<code>jac</code> / 10^{-8}	Optimizer scaling; <code>max_nfev</code> = 200.
<code>_STALL_WINDOW/_RTOL</code>	12 / 5×10^{-3}	TRF early-stop ($18 / 3 \times 10^{-3}$ under GN).
<code>VAR_FLOOR</code>	10^{-6}	Dupire-extraction local-variance floor.
<code>VOLFIT_LV_PHI_DENSE_MB</code>	512 MB	Dense per-step sensitivity-basis budget; above it the exact row-sparse store (≤ 8 slots per node) takes over (appendix B).
<i>Hidden (separate log-strike validator, <code>pde.py</code>)</i>		
<code>DEFAULT_N_K</code> / <code>DEFAULT_DT_MAX</code>	1201 / 1/400	Validator grid.
<code>N_RANNACHER</code>	4	Validator Rannacher quarter-steps.
<code>SPAN_SD</code> / <code>SPAN_MIN</code>	7.5 / 0.6	Validator grid span (sd / floor).

B Performance notes

Every multiplier below was measured on this repository’s development machine; ratios move with hardware, cache state and chain size, which is why they live here rather than in section 9.

1. **Numba march** (`lvFastKernel`). Factor-once no-pivot Thomas (proposition 3 is why no pivoting), m sensitivity columns as the contiguous inner SIMD loop, fused source: $6.1\text{--}6.9\times$ vs LAPACK banded *march-only*, exact to $\sim 10^{-15}$; graceful banded fallback without Numba. The first attempt (column-outer scalar Thomas) won only $1.2\times$ — the lever was the loop order.
2. **Early stop** (`lvEarlyStop`). Best-iterate stop on option-block stall: $1.45\times$ (SPY, $200 \rightarrow 109$ evaluations) to $3.3\times$ (NVDA, $200 \rightarrow 41$) at a deliberately accepted $+0.10\text{--}0.25$ vol bp drift from the full-cap fit; window 0 is byte-identical.

3. **Matrix-free GN** (`lvSolver=gn`). Preconditioned LSMR on the assembled-Jacobian products of section 8; no dense SVD. 1.3–1.65 $\times$ over TRF at tensor sizes; golden-locked to the TRF optimum within tolerance; automatic TRF fallback; sparse CSR regularization block.
4. **Warm starts**. Previous-surface recalibration $\sim 38\times$ *on a small synthetic test* (qualitatively large on real chains); parametric-Dupire cold seed 1.3–1.8 $\times$; θ_{ref} fixed so the objective is unchanged.
5. **Solver scaling and tolerances**. `x_scale='jac'` plus termination tolerances relaxed $10^{-12} \rightarrow 10^{-8}$ on the TRF path: the fit is governed by quote noise, vega floors and bands, so the tighter tolerance only bought iterations — evaluation count cut, surface identical.
6. **Sensitivity-basis memory guard**. The θ -independent per-step hat basis is naturally one dense $(n_{\text{steps}}, n_{\text{interior}}, m)$ tensor; on large grids it outgrew memory and *crashed* rather than slowed. Above a budget (`VOLFIT_LV_PHI_DENSE_MB`, default 512 MB) the build switches to an exact row-sparse store — at most eight value/column slots per interior node, some 30–60 $\times$ smaller — scattered by a Numba kernel. At or below budget the dense path is byte-identical; the over-budget cases carried no byte-identity obligation, because they used to die.
7. **Shelved** (documented in the perf roadmap): coarse-grid calibration (biases θ by up to ~ 26 vol points — non-viable); the first Numba attempt (wrong loop order, 1.2 $\times$); Rannacher/CN ($\sim 1.1\times$ and a measured coarse- x arbitrage, section 5.2); GN for the non-smooth band objective; thread/process parallelism *within one fit* (GIL-negative). Five dead ends confirming the cost of a single fit is distributed, not localized. *Across* fits the story differs: the background Calibrate ships slice fits and per-ticker LV fits to a process pool (`VOLFIT_CALIB_WORKERS`; serial and pooled runs byte-identical) — parallelism pays between independent solves, not inside one.

Caution (engineering): the field crash that pretended to be a matrix

The one production crash in this machinery was an interface bug, not a math bug: `LinearizedJacobian` deliberately has no `.T` attribute (it is an operator, not a matrix), and the early-stop handler, reached only on the rare GN $\rightarrow$ TRF fallback path that then stalls, called `jac.T @ r` for the final gradient — an `AttributeError` in the field (the R1 backtest finding). The fix dispatches to `apply_jacobian_transpose`, and the exact fallback-then-stall path is now test-locked. Lesson: an object that pretends to be a matrix must either implement the whole contract or make each unsupported member fail at construction, not mid-fit.

C Traceability

Read the anchors for what they lock. The “arbitrage-clean” tests are regression checks on selected surfaces, not proofs of remark 3’s missing convexity theorem; the “adjoint” tests lock the *implemented* matrix products of section 8, not the unbuilt backward recursion; the GN–TRF test compares surfaces within tolerance (the schema’s stated ~ 0.25 vol bp trade), not to identity; the warm-start test locks the unchanged objective plus observed agreement; and the case-file improvements ($108 \rightarrow 23.5, 25.7 \rightarrow 2.6$) are prose evidence from the incident logs — the cited tests lock the *gates*, not those exact numbers.

Table 4: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor <i>Test anchors</i>
P_1 basis: partition of unity; nodal bounds bound the surface (hull)	proposition 2	models/localvol/affine.py test_localvol_affine.py::test_basis_partition_of_unity_and_nodal_interpolation::test_nodal_bounds_imply_surface_bounds
Selected calibrated surfaces measure arbitrage-clean (regression, not proof)	remark 3	models/localvol/affine.py test_localvol_affine.py::test_calibrated_surface_is_arbitrage_free test_api_affine.py::test_affine_density_is_clean_no_interior_zeros
Scheme converges (flat + skew round trips)	section 5	models/localvol/pde.py test_localvol.py::test_spatial_convergence::test_skew_dupire_round_trip
Tangent sensitivities exact vs FD	proposition 4	models/localvol/affine.py test_localvol_affine.py::test_forward_sensitivities_match_finite_differences
Implemented $Jv/J^T w$ products and gradient correct (matrix form, not the PDE adjoint)	section 8	models/localvol/affine_gn.py test_affine_gn.py::test_jacobian_transpose_inner_product_identity::test_gradient_alpha_test
GN matches TRF within tolerance; fallback-then-stall safe (R1)	section 9	models/localvol/affine_gn.py test_affine_gn.py::test_gn_matches_trf_on_golden::test_gn_trf_fallback_then_stall_returns_surface
Numba march = banded march	listing 1	models/localvol/affine_march.py test_affine_march.py::test_numba_march_matches_banded_prices_and_sens::test_numba_calibration_matches_banded_surface
Convex wing confined to the extrapolation tail	section 10	api/affine_fit.py test_affine_grid_design.py::test_convex_wing_confined_to_quoted_extrapolation
Coverage floor / PDE step touch only short fronts	section 10	api/affine_fit.py test_affine_grid_design.py::test_coverage_floor_densifies_only_the_short_front::test_pde_dx_refines_only_short_surfaces
Warm start: objective unchanged, evals cut	section 9	api/affine_fit.py test_affine_warm_start.py::test_recalibration_warm_starts_and_cuts_evals

Claim	Object	Code anchor <i>Test anchors</i>
Source-PDE var swap + free left slope	section 6	models/localvol/varswap_pde.py test_varswap_source.py test_affine_grid_design.py::test_free_a_ reduces_varswap_error
Diagnostics are a pure side-channel	section 10	api/affine_diag.py test_affine_diag.py::test_record_shape_and_ purity
Dupire extraction returns NaN on arbitrage	appendix D	models/localvol/dupire.py test_localvol.py::test_dupire_arbitrage_ returns_nan
Prior operators enter as signed baskets	section 7	api/affine_fit.py, affine_calib.py test_affine_basket.py::test_basket_pulls_ surface_toward_target ::test_empty_baskets_byte_identical

D Reference implementation: Dupire extraction

Forward calibration never differentiates the data, but the *inverse* map — reading a local-variance grid off an already-fitted implied total-variance surface $w(k, \tau)$ — is Gatheral’s formula [3], used for grid export, the parametric cold seed of section 9, the graph LV projection (Note 14), and panel A of fig. 1. The denominator g is the butterfly function: $g \leq 0$ marks strike arbitrage in the implied surface, where no positive local variance can reproduce it — returned as `nan`, never clipped. Executed against production before committing: matches `models/localvol/dupire.py` to 10^{-13} .

Listing 2: Dupire/Gatheral local variance from an implied total-variance surface (distilled from `models/localvol/dupire.py`).

```

1 def dupire_local_variance(k, w, wk, wkk, wt):
2     r = k/w                                # w = w(k, tau), with k/kk/tau
3     g = 1 - r*wk + 0.25*(-0.25 - 1/w + r*r)*wk*wk + 0.5*wkk
4     return np.where((w > 0) & (g > 0),      # g <= 0: butterfly arbitrage
5         -> nan
6         wt/np.where(g > 0, g, 1.0), np.nan)
```

References

- [1] B. Dupire. Pricing with a smile. *Risk*, 7(1):18–20, 1994.
- [2] J. Andreasen and B. Huge. Volatility interpolation. *Risk*, March 2011.
- [3] J. Gatheral. *The Volatility Surface*. Wiley, 2006.
- [4] M. Giles and R. Carter. Convergence analysis of Crank–Nicolson and Rannacher time-marching. *J. Comput. Finance*, 9(4), 2006.

- [5] A. Berman and R. Plemmons. *Nonnegative Matrices in the Mathematical Sciences*. SIAM, 1994.